2026 16th International Conference on System Engineering & Technology (ICSET)
3-4 November 2026, Bandung, Indonesia

Physics-Informed Machine Learning for Parameter Estimation in Rigid Multi-Quadrotor Transport System

Miqdad Abdurrahman, Anggera Bayuwindra, Hilton Tnunay
Sekolah Teknik Elektro dan Informatika (STEI)
Institut Teknologi Bandung (ITB)
Bandung, Indonesia
Email: 23224011@mahasiswa.itb.ac.id, bayuwindra@itb.ac.id, hilton.tnunay@itb.ac.id

***Abstract*—Cooperative multi-quadrotor systems rigidly attached to a common frame provide scalable solutions for heavy-lift transportation. However, unmodeled payload mass and unknown spatial offsets can introduce continuous disturbance torques, degrading controller performance and causing lateral drift. This paper introduces a fixed time-step simulation of a Physics-Informed Neural Network (PINN) parameter estimation framework to dynamically identify unmodeled, off-center payloads. To achieve online parameter adaptation, the proposed PINN utilizes continuous gradient descent over a short-term receding data horizon. By embedding translational and rotational physics residuals derived from Euler-Lagrange and Euler-Poincaré equations, the network enforces globally valid rigid-body constraints. The estimated parameters are then continuously fed into the system's control allocation matrix, enabling immediate and adaptive thrust reallocation. Simulations confirm that the integrated PINN estimator converges the desired parameters, arrests transient lateral drift, and successfully restores trajectory tracking stability during dynamic flight maneuvers.** [1]

***Index Terms*—Multi-Agent Systems, Physics-Informed Neural Networks, Cooperative Payload Transport, Parameter Estimation, Unmanned Aerial Vehicles.**

I. Introduction

A. Motivation and Related Works

Cooperative multi-quadrotor systems provide scalable, high-capacity solutions for payload transportation [1], [2]. While cable-suspended payloads [3], [4] are common, rigidly attaching quadrotors to a common frame allows the distributed system to be controlled as a single composite rigid body [2]. This simplification assumes a symmetrical mass distribution [5], [6]; in practice, uncertain payload Centers of Gravity (CoG) introduce unmodeled disturbance torques that cause severe lateral drift and may lead to a crash [7], [8].

To mitigate these disturbances, the literature has explored structural hardware optimizations such as canting rotors to increase yaw authority [2], alongside several algorithmic adaptations. These include classical batch and recursive least-squares [5], Concurrent Learning (CL) frameworks [7], and Digital Twin (DT) Model Predictive Control (MPC) [8]. Building on this body of work, emerging machine learning techniques offer an alternative estimation mechanism for real-time flight stabilization.

This work presents a Proof of Concept (PoC) for a Physics-Informed Neural Network (PINN) estimator for cooperative, rigid multi-quadrotor payload transport in simulation. We assess the feasibility of using a PINN to mitigate disturbances from an unmodeled, off-center static payload. The estimator identifies unknown payload mass and static spatial shifts over a short-term receding data horizon, driven by an energy-based loss derived from the Euler-Lagrange and Euler-Poincare equations. It is integrated with a small-angle multi-agent controller [5] that receives the updated offsets directly in its control allocation matrix.

II. Preliminaries

A. Physics-Informed Parameter Estimation

Physics-Informed Neural Networks (PINNs) embed known physical laws into the deep learning optimization process. Unlike purely data-driven models, PINNs treat the governing Ordinary or Partial Differential Equations as explicit regularizers within the loss function [9], [10].

The network consists of fully connected layers $\mathcal{N}$ parameterized by weights and biases θ [9]. Consider a dynamical system governed by $\mathcal{F}(u, \dot{u}, \ddot{u}, t) = 0$. The PINN acts as a surrogate mapping time t to a predicted state $\hat{u}(t)$:

$$\mathcal{N}_\theta(t) = \hat{u}(t). \tag{1}$$

Training minimizes a composite loss $\mathcal{L}_{total}$ balancing empirical observation against physical consistency. The data loss anchors the network to measurements via the Mean Squared Error (MSE) over n time steps t_i:

$$\mathcal{L}_{data} = \lambda_1 \frac{1}{n} \sum_{i=1}^{n} \left\| \hat{u}(t_i) - u_{true}(t_i) \right\|^2. \tag{2}$$

Simultaneously, the network evaluates the physical validity of its predictions. Rather than relying on numerical approximation, PINNs leverage Automatic Differentiation (Autograd) to

[1]The source code for the PINN estimator and simulation environment is available at: https://github.com/miqdude/pinn_invese_estimator_conf_repo

XXX-X-XXXX-XXXX-X/XX/$XX.00 ©20XX IEEE

compute exact analytical derivatives of the output with respect to the input (e.g., $\dot{\hat{u}} = \frac{\partial \mathcal{N}_\theta}{\partial t}$). These derivatives are substituted back into the governing equation to form a physics residual $R(t) = \mathcal{F}(\hat{u}, \dot{\hat{u}}, \ddot{\hat{u}}, t)$. The physics loss ($\mathcal{L}_{physics}$) penalizes any deviation across n collocation points:

$$\mathcal{L}_{physics} = \lambda_2 \frac{1}{n} \sum_{j=1}^{n} \|R(t_j)\|^2 . \tag{3}$$

The total loss is calculated as a sum of these objectives, $\mathcal{L}_{total} = \mathcal{L}_{data} + \mathcal{L}_{physics}$. By minimizing this composite function, the network learns to predict states that not only match the data but also strictly obey the underlying physical mechanics of the system [10].

III. System Description

A. System Modeling

We define an inertial world frame $\mathcal{W}$ with an upward-pointing Z-axis, and a body frame $\mathcal{B}$ anchored at the board's geometric center. Within $\mathcal{B}$, the positive X-axis points forward along the vehicle, the positive Y-axis points to the left, completing a right-handed coordinate system, and the positive Z-axis points upward. At identity orientation (level and unrotated), the axes of $\mathcal{B}$ align with those of $\mathcal{W}$.

The rigid-frame vehicle consists of a 0.8 kg, 1×1 m square board with four 1.5 kg Iris quadrotors [11] rigidly mounted at its corners. The four quadrotors are indexed $i \in \{1, 2, 3, 4\}$ and mounted at the board's corners, with quadrotor i at positions $(x_1, y_1) = (0.5, 0.5)$, $(x_2, y_2) = (0.5, -0.5)$, $(x_3, y_3) = (-0.5, 0.5)$, and $(x_4, y_4) = (-0.5, -0.5)$ in $\mathcal{B}$. Each quadrotor i maintains a local frame $\mathcal{B}_i$ that is strictly parallel to $\mathcal{B}$: because the attachment is rigid, there is no relative rotation between $\mathcal{B}_i$ and $\mathcal{B}$, and the two frames differ only by a fixed translation (x_i, y_i) in the board's XY plane.

The vehicle's global position in $\mathcal{W}$ is denoted by $\mathbf{r} = [x, y, z]^T$, and its orientation is parameterized by the Euler angles $\boldsymbol{\eta} = [\phi, \theta, \psi]^T$.

The MAS employs a centralized control architecture derived from [5]. The composite rigid body is commanded via a unified control wrench $\boldsymbol{\tau}$, defined as:

$$\boldsymbol{\tau} = [F_B, M_{xB}, M_{yB}, M_{zB}]^T, \tag{4}$$

where F_B is the total collective thrust along the board's Z-axis, and M_{xB}, M_{yB}, and M_{zB} are the desired roll, pitch, and yaw moments applied to the board frame $\mathcal{B}$. These relate to the individual quadrotor thrust inputs $\mathbf{u} = [F_1, F_2, F_3, F_4]^T$, where F_i denotes the thrust produced by quadrotor i along its local Z-axis (parallel to the board's Z-axis), through the allocation matrix $A \in \mathbb{R}^{4\times4}$:

$$\boldsymbol{\tau} = A\mathbf{u}, \tag{5}$$

$$A = \begin{bmatrix} 1 & 1 & 1 & 1 \\ y_1 & y_2 & y_3 & y_4 \\ -x_1 & -x_2 & -x_3 & -x_4 \\ \gamma_1 & \gamma_2 & \gamma_3 & \gamma_4 \end{bmatrix}. \tag{6}$$

The rows of A correspond to the total thrust, roll moment, pitch moment, and yaw moment on the board, respectively. Here, (x_i, y_i) is the position of quadrotor i's center of thrust expressed in $\mathcal{B}$, and γ_i is the yaw moment coefficient (torque-to-thrust ratio) of quadrotor i, with its sign dictated by the rotor's spin direction.

B. Dynamic Thrust Allocation

When a payload shifts from the board's geometric center, it alters the effective moment arms of each quadrotor. Let $\hat{\mathbf{c}} = [\hat{c}_x, \hat{c}_y]^T$ denote the estimated system CoG expressed in $\mathcal{B}$, produced directly by the PINN and fed to the control allocation matrix. Redefining the lever arms of quadrotor i, measured from the system CoG, as $\Delta x_i = x_i - \hat{c}_x$ and $\Delta y_i = y_i - \hat{c}_y$, the allocation matrix adapts dynamically:

$$\mathbf{A}_{\text{final}} = \begin{bmatrix} 1 & 1 & 1 & 1 \\ \Delta y_1 & \Delta y_2 & \Delta y_3 & \Delta y_4 \\ -\Delta x_1 & -\Delta x_2 & -\Delta x_3 & -\Delta x_4 \\ \gamma_1 & \gamma_2 & \gamma_3 & \gamma_4 \end{bmatrix}. \tag{7}$$

Control inputs are computed via the Moore-Penrose pseudo-inverse to handle transient rank deficiencies:

$$\mathbf{u} = \mathbf{A}_{\text{final}}^{+} \boldsymbol{\tau}. \tag{8}$$

C. Centralized Control Design

Based on [5], the controller separates position and attitude tracking. Position and velocity errors define commanded acceleration $\ddot{\mathbf{r}}_{cmd}$:

$$\mathbf{e}_p = \mathbf{r}_{des} - \mathbf{r}, \quad \mathbf{e}_v = \dot{\mathbf{r}}_{des} - \dot{\mathbf{r}}, \tag{9}$$

$$\ddot{\mathbf{r}}_{cmd} = K_p \mathbf{e}_p + K_d \mathbf{e}_v + \ddot{\mathbf{r}}_{\text{ff}}. \tag{10}$$

Desired roll (ϕ^{des}), pitch (θ^{des}), and total thrust (F_B^{des}) are extracted via small-angle approximation:

$$\begin{aligned} \phi^{des} &= \frac{1}{g}(\ddot{r}_x^{des} \sin\psi_T - \ddot{r}_y^{des} \cos\psi_T), \\ \theta^{des} &= \frac{1}{g}(\ddot{r}_y^{des} \cos\psi_T + \ddot{r}_y^{des} \sin\psi_T), \\ F_B^{des} &= m(\ddot{r}_z^{des} + g). \end{aligned} \tag{11}$$

Inner-loop desired moments $\mathbf{M}_B^{des}$ regulate orientation against angular velocities (p, q, r):

$$\begin{aligned} M_{xB}^{des} &= k_{p,\phi}(\phi^{des} - \phi) + k_{d,\phi}(p^{des} - p), \\ M_{yB}^{des} &= k_{p,\theta}(\theta^{des} - \theta) + k_{d,\theta}(q^{des} - q), \\ M_{zB}^{des} &= k_{p,\psi}(\psi^{des} - \psi) + k_{d,\psi}(r^{des} - r). \end{aligned} \tag{12}$$

IV. Proposed PINN-Based Estimator Design

A. PINN Architecture

To address the 6-DOF rigid-body dynamics, our PINN framework utilizes continuous time t as the sole independent variable to solve the time-dependent ODEs. The output is a 12-dimensional state vector consisting of posisition, linear velocity, attitude, and angular velocity [10], [12], [13]. The PINN design used in this research is described in Table I.

TABLE I
Proposed Fully Connected layer PINN Architecture Specifications

Layer	Number of Neurons	Activation Function
Input	1 (Continuous time t)	None
Hidden 1	64	tanh
Hidden 2	64	tanh
Output	12 (Kinematic state vector)	Linear

B. Loss Function Formulation

The neural network acts as a surrogate model mapping continuous time t to the predicted 12 states of the system. We define the complete state vector $\mathbf{s}$, comprising the position, linear velocity, attitude, and angular velocity:

$$\mathbf{s} = [\mathbf{r}^T, \dot{\mathbf{r}}^T, \boldsymbol{\eta}^T, \boldsymbol{\omega}^T]^T. \tag{13}$$

To train the network, we formulate a composite loss function $\mathcal{L}_{total}$ [13], [14] that simultaneously penalizes empirical data deviations and violations of rigid-body dynamics. This objective function is defined as a weighted sum of the data-driven observation loss ($\mathcal{L}_{data}$), the translational physics residual ($\mathcal{L}_{trans}$), and the rotational physics residual ($\mathcal{L}_{rot}$):

$$\mathcal{L}_{total} = \lambda_1 \mathcal{L}_{data} + \lambda_2 \mathcal{L}_{trans} + \lambda_3 \mathcal{L}_{rot}, \tag{14}$$

where the hyperparameters λ_1, λ_2, and λ_3 balance the relative contribution of each loss term. These weights are held fixed during each training run, and their effect on estimator performance is studied in Section V.

The data loss ensures the network's predictions ($\hat{s}$) anchor to the physical reality measured by the onboard sensors. It is computed as the Mean Squared Error (MSE) over a temporal window of n collocation points:

$$\mathcal{L}_{data} = \frac{1}{n} \sum_{i=1}^{n} \left\| \hat{s}_i - s_{true,i} \right\|^2. \tag{15}$$

To enforce the translational physical constraints, we formulate a residual loss based on the Euler-Lagrange equations [15]. Let $L = T_{trans} - V$ denote the system Lagrangian, where T_{trans} is the kinetic energy and V is the potential energy. The translational dynamics must satisfy:

$$\frac{d}{dt}\left(\frac{\partial L}{\partial \mathbf{v}}\right) - \frac{\partial L}{\partial \mathbf{r}} = \mathbf{F}_{ext}. \tag{16}$$

Evaluating the kinetic inertial term yields $\hat{m}\dot{\hat{\mathbf{v}}}$. Rather than analytically deriving the potential gradient, we leverage PyTorch's Automatic Differentiation (Autograd) engine to dynamically compute $\frac{\partial V}{\partial \mathbf{r}}$ during the forward pass. The resulting translational physics loss minimizes the residual of these forces across the temporal window:

$$\mathcal{L}_{trans} = \frac{1}{n} \sum_{i=1}^{n} \left\| \hat{m}\dot{\hat{\mathbf{v}}}_i + \frac{\partial V}{\partial \hat{\mathbf{r}}_i} - \mathbf{F}_{ext,i} \right\|^2. \tag{17}$$

While translational dynamics are straightforward, evaluating rotational dynamics using minimal coordinates (e.g., Euler angles) introduces mathematical singularities, commonly known as gimbal lock, at extreme pitch or roll angles. To ensure global mathematical validity, we evaluate the rotational dynamics directly on the Special Orthogonal group, $SO(3)$. The rotational kinetic energy of the rigid multi-agent system is defined as:

$$T_{rot} = \frac{1}{2} \hat{\boldsymbol{\omega}}^T \hat{\mathbf{J}} \hat{\boldsymbol{\omega}}, \tag{18}$$

where $\hat{\mathbf{J}} \in \mathbb{R}^{3\times3}$ is the estimated inertia tensor. By applying the Euler-Poincaré equations of motion for rigid body we get:

$$\frac{d}{dt}\left(\frac{\partial T_{rot}}{\partial \hat{\boldsymbol{\omega}}}\right) + \hat{\boldsymbol{\omega}} \times \frac{\partial T_{rot}}{\partial \hat{\boldsymbol{\omega}}} = \boldsymbol{\tau}_{ext}. \tag{19}$$

The partial derivative $\frac{\partial T_{rot}}{\partial \boldsymbol{\omega}} = \hat{\mathbf{J}}\hat{\boldsymbol{\omega}}$ represents the system's angular momentum, Eq. (19) resolves into the Newton-Euler formulation for rigid body rotation. In our cooperative framework, the total external torque $\boldsymbol{\tau}_{ext}$ comprises the known applied control wrench $\boldsymbol{\tau}_{app}$ and the unknown disturbance torque $\boldsymbol{\tau}_{dist}$ generated by the payload's CoG offset. The rotational physics residual evaluated during the forward pass is thus formulated as:

$$\mathcal{R}_{rot} = \left(\hat{\mathbf{J}}\dot{\hat{\boldsymbol{\omega}}} + \hat{\boldsymbol{\omega}} \times (\hat{\mathbf{J}}\hat{\boldsymbol{\omega}})\right) - (\boldsymbol{\tau}_{app} - \boldsymbol{\tau}_{dist}). \tag{20}$$

Finally, the rotational loss penalizes the mean squared error of this residual, forcing the neural network to implicitly map the physical discrepancies to the true disturbance parameters:

$$\mathcal{L}_{rot} = \frac{1}{n} \sum_{i=1}^{n} \left\| \mathcal{R}_{rot,i} \right\|^2. \tag{21}$$

C. Simulation Design

To eliminate computational latency anomalies and isolate the convergence behavior of the online-learning PINN estimator, a synchronous fixed-time simulation was utilized for this proof of concept. This guarantees temporal alignment between the plant state and the automatic differentiation tracking within the physics loss. Transitioning these mechanics to an asynchronous, dual-rate Software-in-the-Loop (SIL) framework to handle real-world computational lag is preserved for future work.

In Algorithm 1, the PINN estimates the system CoG $\hat{\mathbf{c}} = [\hat{c}_x, \hat{c}_y]^T$ and the total mass $\hat{m}$. The new system inertia $\hat{\mathbf{J}}$ is

not estimated directly; it is derived from the estimated payload offset $\hat{\mathbf{d}}$, the mass $\hat{m}$, and the intial system's inertia $\mathbf{J}$ via the parallel-axis theorem.

Algorithm 1 Fixed Time-step Online Parameter Estimation

Require: Simulation duration $T_{\max}$, physics time step Δt
Require: PINN optimization interval N steps
Require: PINN gradient steps per cycle S_{steps}
Require: Replay buffer B with capacity C
Require: Initial parameter guesses: $\hat{m}, \hat{c}_x, \hat{c}_y$
Initialize quadrotor physical states $\mathbf{x}_0$
Initialize PINN $\mathcal{N}_\theta$ with random weights θ
$t \leftarrow 0, \quad k \leftarrow 0$
while $t \leq T_{\max}$ **do**
  Read current state $\mathbf{s}_t = (\mathbf{r}, \dot{\mathbf{r}}, \boldsymbol{\eta}, \boldsymbol{\omega})$
  Compute target states $\mathbf{p}_d, \mathbf{v}_d, \mathbf{a}_d, \psi_d, \dot{\psi}_d$ at t
  Append $\mathbf{s}_t$ and applied thrusts $\mathbf{F}_t$ to buffer B
  if $k \bmod N = 0$ **and** $|B| = C$ **then**
    Compute composite loss $\mathcal{L}_{\text{total}}$ over B
    Optimize θ via backpropagation for S_{steps} steps
    Extract $\hat{m}, \hat{c}_x, \hat{c}_y$ from $\mathcal{N}_\theta$
    Compute payload offset $\hat{\mathbf{d}} = \frac{m_f + \hat{m}_p}{\hat{m}_p}[\hat{c}_x, \hat{c}_y]^T$
    Calculate $\hat{\mathbf{J}}$ from $\hat{m}, \hat{\mathbf{d}}, \mathbf{J}$
  end if
  Compute desired control wrench $(\mathbf{F}_{\text{des}}, \mathbf{M}_{\text{des}})$
  Construct $\mathbf{A}_{\text{final}}$ using $\hat{c}_x, \hat{c}_y$
  $\mathbf{u} \leftarrow \mathbf{A}^+_{\text{final}}[\mathbf{F}^T_{\text{des}}, \mathbf{M}^T_{\text{des}}]^T$
  Apply saturated commands $\mathbf{u}$ to physics engine
  $t \leftarrow t + \Delta t, \quad k \leftarrow k + 1$
end while

V. Validation and Results

A. Simulation Environment and Setup

Simulations were conducted on a laptop equipped with an NVIDIA RTX 3050 GPU (6 GB VRAM) using Python 3.11.0 with CUDA support. Table II contains the independent variables used in Algorithm 1.

TABLE II
System and Simulation Parameters

Parameter	Symbol	Value
Simulation Time-Step	Δt	$1/240$ s
Gravitational Acceleration	g	9.81 m/s^2
Simulation Duration	T_{max}	90 s
Base Frame Mass	-	6.8 kg
Quadrotor Mass	-	1.5 kg
Payload Mass	-	1.0 kg
Payload Volume	-	125×10^{-6} m^3
Total System Mass	m	7.8 kg
Base System Inertia on X axis	J_{xx}	1.713
Base System Inertia on Y axis	J_{yy}	1.713
Base System Inertia on Z axis	J_{zz}	3.413
Max Thrust per Drone	-	50 N
Unknown Payload Offset	d_x, d_y	$(0.1, 0.1)$ m
PINN Window Size	C	50 steps
Optimization Frequency	N	8 steps
Gradient Steps per Cycle	S_{steps}	3 steps

Network optimization is driven by the Adam optimizer with a learning rate of 5×10^{-3} [14]. To systematically evaluate the learning characteristics of the PINN, various hyperparameters (λ) for the composite physics loss were tested.

The experimental flight sequence initiates at $t = 0$ s with a vertical ascent to a baseline hover. The system stabilizes at an altitude of 3 meters for an initial 5-second window, allowing the physical state to settle before dynamic maneuvers commence. To excite the system's dynamics and test the estimator, the frame is subjected to three distinct spatial trajectories: a figure-8, a circular path, and a uniaxial "lawnmower" motion.

For the sake of brevity in the manuscript, we define only the desired 3D position vector $\mathbf{p}_d(t) = [x_d, y_d, z_d]^T \in \mathbb{R}^3$ for each maneuver. The corresponding feedforward velocity $\mathbf{v}_d(t)$ and acceleration $\mathbf{a}_d(t)$ setpoints utilized by the geometric controller are computed via exact analytical time differentiation of $\mathbf{p}_d(t)$.

The figure-eight trajectory incorporates a constant radius ($R = 8$ m), an angular frequency ($\omega = 0.10$ rad/s), and a sinusoidal vertical variation (z_v) to simultaneously explore translation on frame's XYZ axis:

$$\mathbf{p}_d(t) = \begin{bmatrix} R\sin(\omega t) \\ \frac{R}{2}\sin(2\omega t) \\ z_0 + z_v \sin(\omega t) \end{bmatrix}. \tag{22}$$

The Circular trajectory features a radius ($R = 5$ m) and angular frequency ($\omega = 0.15$ rad/s), retaining the continuous altitude variation:

$$\mathbf{p}_d(t) = \begin{bmatrix} R\sin(\omega t) \\ R(1 - \cos(\omega t)) \\ z_0 + z_v \sin(\omega t) \end{bmatrix}. \tag{23}$$

The Lawnmower maneuver isolates longitudinal dynamics by commanding forward-backward motion exclusively along the world X-axis at an elevated, constant altitude of $z_0 = 5$ m. For a single maneuver phase with an amplitude $A = 2.0$ m and duration $T_{phase} = 5.0$ s, the trajectory is defined as:

$$\mathbf{p}d(t) = \begin{bmatrix} \frac{A}{2}(1 - \cos(\omega_L t)) \\ 0 \\ z_0 \end{bmatrix}, \tag{24}$$

where $\omega_L = \frac{2\pi}{Tphase}$ represents the governing phase angular frequency.

B. Parameter Convergence and Estimation

The trajectory tracking results in Fig. 1 demonstrate the significance of embedding physical constraints into the neural network's loss function. As shown in the top row, the data-only baseline fails to follow the reference path and exhibits significant drift across all three maneuvers. The bottom row shows that the physics-informed configuration tracks each reference trajectory tightly. By embedding the system's dynamic model into the loss, the estimator compensates for the disturbance introduced by the payload offset, producing more stable tracking.

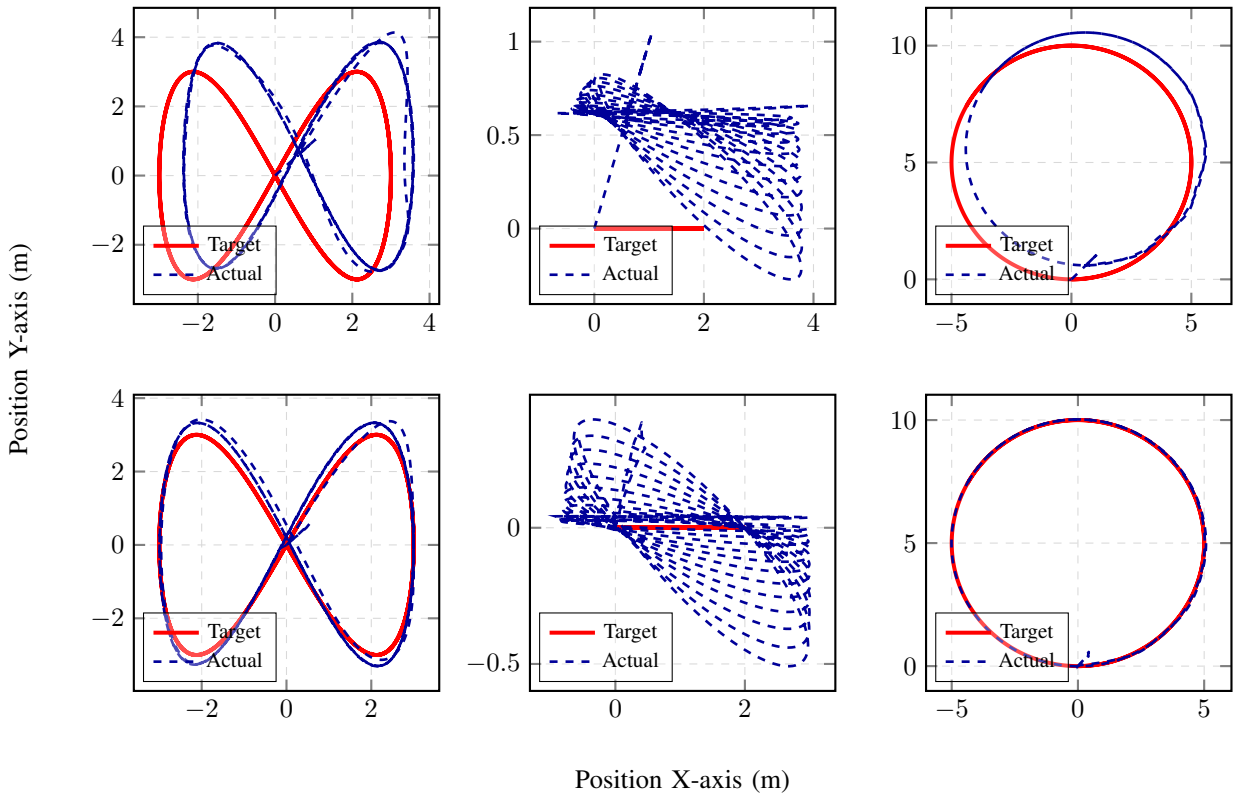


Fig. 1. Trajectory tracking performance in the XY plane for each trajectory divided by columns in order figure-eight, lawnmower, and circular path. The top row illustrates the network performance with 0 weight of phyiscs loss, while the bottom row demonstrates the improved trajectory using the proposed method.

TABLE III
STEADY-STATE ESTIMATION ERRORS OVER THE LAST 20 S ACROSS TRAJECTORIES AND λ CONFIGURATIONS.

No	Trajectory	$\lambda_{1,2,3}$	$\hat{m}$	$\hat{d}_x$	$\hat{d}_y$	$\hat{J}_{xx}$	$\hat{J}_{yy}$	$\hat{J}_{zz}$	Trajectory RMSE (m)
1	Figure 8	1,1,1	7.834	0.0489	0.0488	1.7181	1.7180	3.4179	0.3650
2	Figure 8	1,0,0	6.801	0.0000	0.0000	1.7131	1.7131	3.4130	0.8863
3	Figure 8	1,1,0	7.836	0.0000	0.0000	1.7156	1.7156	3.4130	0.8863
4	Figure 8	1,0,1	6.801	1.7180	1.7184	1.8617	1.8610	3.7095	0.8528
5	Figure 8	0.1,1,1	7.827	0.0492	0.0490	1.7180	1.7181	3.4180	0.3637
6	Figure 8	10,1,1	7.838	0.0487	0.0487	1.7181	1.7181	3.4180	0.3599
7	Lawn	1,1,1	7.980	0.0433	0.0448	1.7183	1.7189	3.4184	0.64389
8	Lawn	1,0,0	6.801	0.0000	0.0000	1.7131	1.7131	3.4130	1.1394
9	Circle	1,1,1	7.770	0.0513	0.0514	1.7180	1.7180	3.4181	0.2315
10	Circle	1,0,0	6.801	0.0000	0.0000	1.7131	1.7131	3.4130	0.8528

Table III quantifies this gap. With the physics terms disabled ($\lambda_{1,2,3} = 1, 0, 0$), the trajectory RMSE reaches 1.1394 m on the lawnmower path and 0.8863 m on the figure-eight. Enabling the full physics loss ($\lambda_{1,2,3} = 1, 1, 1$) reduces these to 0.6439 m and 0.3650 m, respectively, and achieves 0.2315 m on the circular trajectory. The data-only configuration also fails to update any physical parameter: the mass remains at its initial value (6.801 kg) and the payload offsets stay at 0.0000 m across every trajectory, since the physical parameters enter the loss only through the physics residuals. The physics-informed estimator activates these parameters and drives the system toward stable tracking, though it does not recover the offsets exactly—the steady-state values plateau near 0.05 m versus the true 0.10 m. We attribute this gap to limited excitation of the tilt dynamics in the tested trajectories, which weakens the moment-arm signal used to identify the payload offset; the estimator instead settles on a value that satisfies the composite dynamic residual at the cost of some offset accuracy. Note that the system inertia is not a directly estimated parameter but is derived from $\hat{m}$, $\hat{d}_x$, and $\hat{d}_y$ via the parallel-axis theorem; the corresponding columns in Table III therefore reflect the accuracy of the mass and payload-offset estimates rather than independent inertia identification. Configurations 5 and 6, which sweep the data weight by one order of magnitude around the baseline, produce estimates within a few percent of the baseline, indicating that the estimator is not sensitive to moderate reweighting of the loss terms over this range.

The temporal dynamics of this process are detailed in Fig. 2.

The payload offsets converge within the first 20 s across all trajectories but plateau below the target lines, visually confirming the steady-state gap in Table III. The mass estimate adapts more gradually, converging toward the target and stabilizing between 50 s and 60 s. The lawnmower trajectory induces a slight overshoot in the mass estimate, whereas the figure-eight and circular paths maintain more stable convergence during steady state.

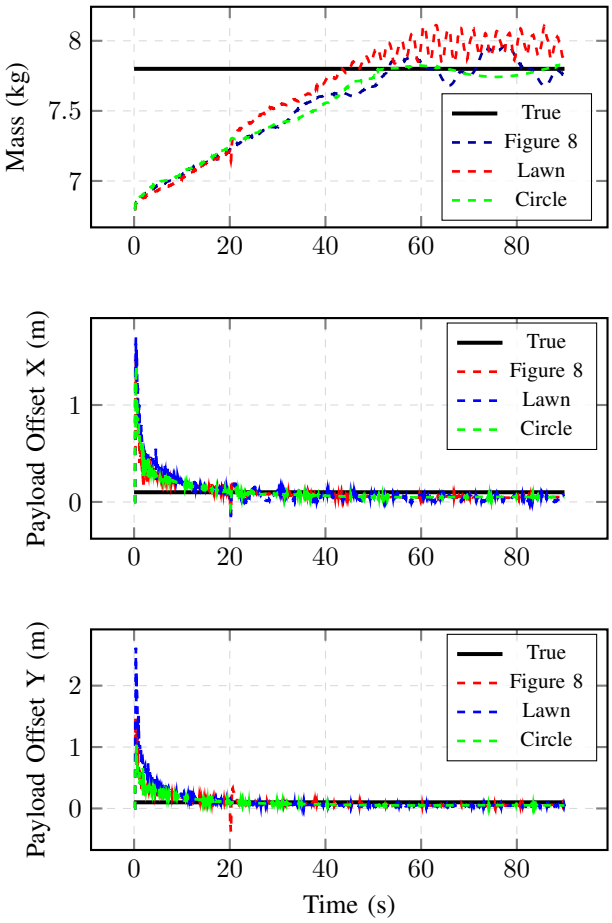


Fig. 2. Parameter convergence for mass, payload offset X, and payload offset Y across all evaluated trajectories.

VI. Conclusion and Future Work

This paper introduced a simulation of online parameter estimation framework for rigid multi-quadrotor payload transportation, driven by a Physics-Informed Neural Network (PINN) estimator. By embedding Euler-Lagrange and Euler-Poincaré equations into the loss function, the network successfully converges on its estimates proving that the network cannot rely solely on the empirical data.

Simulation results validate that integrating this PINN-based estimator into the control allocation matrix enables the system to identify unmodeled payload mass and center-of-mass offsets. The approach effectively arrests transient lateral drift and maintains stable trajectory tracking within a single flight for all tested trajectories. Furthermore, the results indicate that even when absolute parameter convergence is incomplete, the resulting dynamic thrust reallocation is sufficient to restore the baseline controller's authority and counteract unmodeled disturbance torques.

Future work will extend this methodology to software-in-the-loop (SITL) simulation by incorporating a publish-subscribe messaging architecture to connect the controller, the physics engine, and the online estimator. Ultimately, validating this distributed architecture will serve as the foundational step to transition this research from simulation to real-world flight testing.

References

[1] R. Guebsi, S. Mami, and K. Chokmani, "Drones in Precision Agriculture: A Comprehensive Review of Applications, Technologies, and Challenges," *Drones*, vol. 8, no. 11, p. 686, Nov. 2024.

[2] Y. H. Tan, S. Lai, K. Wang, and B. M. Chen, "Cooperative Heavy Lifting Using Unmanned Multi-Agent Systems," in *2018 IEEE 14th International Conference on Control and Automation (ICCA)*. Anchorage, AK: IEEE, Jun. 2018, pp. 1119–1126.

[3] J. Zeng, A. M. Gimenez, E. Vinitsky, J. Alonso-Mora, and S. Sun, "Decentralized Aerial Manipulation of a Cable-Suspended Load using Multi-Agent Reinforcement Learning," Nov. 2025, arXiv:2508.01522 [cs].

[4] B. Peng, B. Su, K. Yi, H. Wu, and D. He, "Modeling and Analysis of Four-UAV-Slung-Load System," in *2023 35th Chinese Control and Decision Conference (CCDC)*. Yichang, China: IEEE, May 2023, pp. 4289–4294.

[5] D. Mellinger, M. Shomin, N. Michael, and V. Kumar, "Cooperative grasping and transport using multiple quadrotors," in *Distributed Autonomous Robotic Systems*. Springer, 2013, pp. 545–558.

[6] H. Ye, H. Yu, B. Liu, J. Han, Y. Fang, and X. Liang, "Nonlinear Robust Control for Dual-UAV Collaborative Aerial Transportation System," in *2023 7th Asian Conference on Artificial Intelligence Technology (ACAIT)*. Jiaxing, China: IEEE, Nov. 2023, pp. 870–875.

[7] C.-A. Lee and T.-H. Cheng, "Concurrent Learning for Cooperative UAV Transportation of Unknown Payloads," *IEEE Open Journal of Control Systems*, vol. 4, pp. 187–198, 2025.

[8] U. Farid, B. Khan, S. M. Ali, and Z. Ullah, "A Digital Twin Model for UAV Control to Lift Irregular-Shaped Payloads Using Robust Model Predictive Control," *Machines*, vol. 13, no. 11, p. 1069, Nov. 2025.

[9] M. Raissi, P. Perdikaris, and G. E. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational physics*, vol. 378, pp. 686–707, 2019.

[10] R. Mojgani, M. Balajewicz, and P. Hassanzadeh, "Kolmogorov n–width and Lagrangian physics-informed neural networks: A causality-conforming manifold for convection-dominated PDEs," *Computer Methods in Applied Mechanics and Engineering*, vol. 404, p. 115810, Feb. 2023.

[11] F. Furrer, M. Burri, M. Achtelik, and R. Siegwart, *Robot Operating System (ROS): The Complete Reference (Volume 1)*. Cham: Springer International Publishing, 2016, ch. RotorS—A Modular Gazebo MAV Simulator Framework, pp. 595–625.

[12] S. Lakshminarayana, S. Sthapit, and C. Maple, "Application of Physics-Informed Machine Learning Techniques for Power Grid Parameter Estimation," *Sustainability*, vol. 14, no. 4, p. 2051, Feb. 2022.

[13] M. Raissi, P. Perdikaris, and G. Karniadakis, "Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations," *Journal of Computational Physics*, vol. 378, pp. 686–707, Feb. 2019.

[14] G. Serrano, M. Jacinto, J. Ribeiro-Gomes, J. Pinto, B. J. Guerreiro, A. Bernardino, and R. Cunha, "Physics-Informed Neural Network for Multirotor Slung Load Systems Modeling," May 2024, arXiv:2405.09428 [cs].

[15] G. Haghighatdoost, "Euler Poincare Dynamics and Control on Lie Groupoids," May 2026, arXiv:2509.20044 [math.DS].